

Verify Leads — Python File Explanation

Step-by-step technical breakdown of the `verify_leads.py` execution and logic script

What This File Is About

`verify_leads.py` is a standalone Python script used to verify scraped leads. It reads lead information from an input CSV, checks different sources and signals, calculates a verification score from 0 to 100, assigns a verification status, and writes the results into an output CSV.

The script uses free services. Its main external source is OpenStreetMap Nominatim. It can also check a lead's website for relevant keywords and determine whether the lead appears to represent Police, Lawyer, Private Detective, Security, or Unknown.

1. FILE HEADER AND PURPOSE

1. The opening documentation describes the file as a standalone verification script for leads.
2. The normal command is to run the file with an input CSV and an output CSV.
3. The input CSV is expected to contain fields such as `agency_name`, `location`, `website`, `phone`, and `contact_email`.
4. For every lead, the script queries OpenStreetMap Nominatim using the lead name and location.
5. If a website is available, the script checks whether it can be reached and searches its page text for relevant keywords.
6. The collected signals are combined into `verification_score`, ranging from 0 to 100.
7. The score is converted into one of three statuses: Verified, Needs Review, or Unverified.
8. The script writes `verification_status`, `verification_score`, `verified_sources`, and `verified_at` to the output CSV.
9. The comments state that the script does not modify the Streamlit application and can be run locally or in a CI job.

2. IMPORT STATEMENTS

10. `argparse` is imported to create command-line options such as `--input`, `--output`, and `--sample`.
11. `pandas` is imported as `pd` for reading, processing, and writing CSV data.
12. `requests` is used to send HTTP requests to websites and the Nominatim service.
13. `time` is used for the delay between Nominatim requests.
14. `datetime` is used to record when a lead was verified.
15. `re` is used for regular expressions, mainly for text normalization, HTML cleanup, token extraction, and phone-number checking.

3. CONFIGURATION VALUES

16. `NOMINATIM_URL` stores the OpenStreetMap Nominatim search endpoint.
17. `USER_AGENT` identifies the script when it sends web requests. The file contains a placeholder email that can be replaced.
18. `NOMINATIM_DELAY` is set to 1.1 seconds so the script waits between Nominatim requests.
19. `KEYWORDS` contains terms related to detectives, investigations, security, lawyers, advocates, police, and surveillance. These words are used as evidence during verification.

4. NORMALIZE_TEXT(VALUE)

- 20. The `normalize_text` function converts a value into a consistent text form.
- 21. If the value is `None`, it returns an empty string.
- 22. Otherwise, it converts the value to a string, replaces repeated whitespace with a single space, removes leading and trailing spaces, and converts the result to lowercase.
- 23. This normalization makes later keyword and text comparisons more consistent.

5. DETECT_PROFESSION(NAME, WEBSITE, PAGE_TEXT)

- 24. This function tries to classify a lead as Police, Lawyer, Private Detective, Security, or Unknown.
- 25. The name, website, and downloaded page text are normalized and combined into one string.
- 26. If there is no usable combined text, the function returns `Unknown`.
- 27. Police keywords are checked first. A match such as police station or police department returns `Police`.
- 28. Lawyer-related keywords are checked next. A match such as advocate, lawyer, attorney, or legal services returns `Lawyer`.
- 29. Detective-related keywords are then checked. Examples include private detective, private investigator, detective agency, surveillance, and investigators.
- 30. Security-related keywords are checked after that, including security services, security agency, and security guards.
- 31. If none of these groups matches, the function returns `Unknown`.
- 32. The order matters because the first matching profession category is returned.

6. FETCH_WEBSITE_TEXT(URL)

- 33. This function downloads the text of a lead's website so it can be inspected.
- 34. If the URL is missing or is not a string, the function returns an empty string.
- 35. If the URL does not begin with `http://` or `https://`, the script adds `http://`.
- 36. `requests.get` sends the HTTP request with the configured User-Agent and an 8-second timeout.
- 37. `raise_for_status` causes unsuccessful HTTP responses to be treated as errors.
- 38. The returned HTML is cleaned by removing script blocks, style blocks, and HTML tags.
- 39. Extra whitespace is reduced and the resulting page text is converted to lowercase.
- 40. If any exception occurs, the function safely returns an empty string instead of stopping the whole script.

7. SEARCH_NOMINATIM(NAME, LOCATION)

- 41. This function searches OpenStreetMap Nominatim for a place matching the lead name and location.
- 42. The query is created as name plus location when a location is available; otherwise only the name is used.
- 43. The request asks Nominatim for JSON results, address details, and up to three results.
- 44. The first result is treated as the best match and returned.
- 45. If there are no results or the request fails, the function returns `None`.

8. CHECK_WEBSITE_FOR_KEYWORDS(URL, NAME)

- 46. This function combines website checking, keyword detection, and profession detection.
- 47. If no usable website is supplied, it returns `False`, an empty keyword list, `Unknown`, and empty page text.
- 48. Otherwise, `fetch_website_text` downloads and cleans the website content.
- 49. Every keyword in `KEYWORDS` is checked against the downloaded page text.
- 50. All matching keywords are stored in the found list.
- 51. `detect_profession` is called using the lead name, website, and page text.
- 52. The function returns whether keywords were found, the matching keywords, the detected profession, and the page text.

9. NAME_SIMILARITY(NAME_A, NAME_B)

- 53. This function provides a small heuristic for comparing two names.
- 54. If either name is empty, the similarity score is 0.0.
- 55. Each name is converted into lowercase alphanumeric tokens using a regular expression.
- 56. The tokens are stored as sets so that common tokens can be identified.
- 57. The intersection of the two token sets contains the words appearing in both names.
- 58. The similarity is calculated as the number of common tokens divided by the larger token-set size.
- 59. The result is a value between 0 and 1. A higher value means greater token overlap.

10. SCORE_LEAD(ROW)

- 60. score_lead is the main verification function. It receives one lead and returns a score, evidence sources, and profession.
- 61. The function starts with score = 0 and an empty sources list.
- 62. It extracts agency_name, location, website, and phone from the row.
- 63. The profession initially starts as Unknown.

11. NOMINATIM VERIFICATION

- 64. search_nominatim is called with the agency name and location.
- 65. The script then waits for 1.1 seconds to respect the configured Nominatim delay.
- 66. If Nominatim returns a result, the returned display name is compared with the lead name.
- 67. If name similarity is at least 0.35, 50 points are added and nominatim_name_match is recorded.
- 68. If the name similarity is below 0.35 but the result contains a relevant keyword, 35 points are added and nominatim_keyword_match is recorded.
- 69. Otherwise, finding a Nominatim result contributes 10 points and nominatim_found is recorded.

12. WEBSITE VERIFICATION

- 70. check_website_for_keywords is called for the lead's website.
- 71. If relevant keywords are found on the website, 30 points are added.
- 72. The matching keywords are stored in the verified sources list.
- 73. If a profession other than Unknown is detected, the profession is also stored as evidence.
- 74. If no keywords are found but a website exists, the script sends a HEAD request.
- 75. A successful website response with a status below 400 contributes 10 points and records website_exists.
- 76. If the website request fails, the script simply continues without adding those points.

13. PHONE NUMBER VERIFICATION

- 77. The script checks whether a phone value exists.
- 78. Non-digit characters are removed from the phone number.
- 79. If at least 7 digits remain, 10 points are added and phone_present is recorded.

14. PROFESSION-BASED SCORE ADJUSTMENT

- 80. If the detected profession is Police, 10 points are subtracted.
- 81. If the detected profession is Lawyer, 5 points are subtracted.
- 82. The comments explain that these categories are retained but receive lower confidence for the intended detective-lead verification.
- 83. The final score is limited to the range 0 to 100.
- 84. score_lead returns the integer score, evidence sources, and detected profession.

15. MAIN(ARGV=NONE)

- 85. main is the function that controls the complete command-line workflow.
- 86. argparse.ArgumentParser creates the command-line parser.
- 87. --input specifies the input CSV and defaults to scraped_leads.csv.
- 88. --output specifies the output CSV and defaults to verified_leads.csv.
- 89. --sample optionally limits processing to the first N rows, which is useful for testing.
- 90. pandas.read_csv reads the input file into a DataFrame.
- 91. If --sample is greater than zero, only the first requested number of rows is copied for processing.

16. OUTPUT COLUMNS

- 92. The script makes sure these columns exist: verification_status, verification_score, verified_sources, verified_at, and profession.
- 93. For every row, the script prints which lead is currently being processed.
- 94. score_lead is called to obtain the score, evidence sources, and profession.
- 95. A score of 70 or more is assigned the status Verified.
- 96. A score from 40 through 69 is assigned the status Needs Review.
- 97. A score below 40 is assigned the status Unverified.
- 98. datetime.datetime.utcnow().isoformat() plus Z records the verification time in UTC format.
- 99. The result for each lead is stored in a results list.
- 100. The results are converted into a DataFrame and combined with the original lead data.
- 101. The final DataFrame is written to the requested output CSV without adding a pandas index.
- 102. The script prints the number of rows written to the output file.

17. FINAL EXECUTION CHECK

- 103. The if __name__ == '__main__': condition checks whether the file is being run directly.
- 104. If it is being run directly, main() is called and the complete verification process starts.
- 105. This structure also allows the functions to be imported from another Python file without automatically running the whole script.

Score Logic Overview

- **Nominatim name match:** +50 points
- **Nominatim keyword match:** +35 points
- **Nominatim result found:** +10 points
- **Website keyword match:** +30 points
- **Reachable website w/o keywords:** +10 points
- **Valid phone number:** +10 points
- **Police / Lawyer:** -10 / -5 points

Verification Status Criteria

- **Verified:** score ≥ 70
- **Needs Review:** $40 \leq \text{score} < 70$
- **Unverified:** score < 40

Note: The final score is always restricted to the range 0–100.

Overall Workflow & Important Note

Input CSV → Read Leads → Search OpenStreetMap → Compare Names → Check Website & Keywords → Detect Profession & Phone → Calculate Score & Status → **Write Output CSV**

The score is a heuristic verification mechanism, not definitive proof that a lead is genuine. A lead marked Verified meets the defined scoring rules within the code.